GitHub Actions：入门指南（Python）

目标：创建一个小的 GitHub Actions 工作流，触发它并理解其结果。本示例使用 Python 和 cars-crud 仓库，但同样的结构也适用于其他语言和框架。

1. 基本概念

工作流是存储在 .github/workflows/ 中的 YAML 文件。当配置的事件发生时，GitHub 就会运行它。

术语	含义
工作流	由一个 .yml 文件描述的完整自动化
事件	启动工作流的东西，例如 push 或 pull_request
作业	在一台机器上运行的一组步骤
运行器	执行作业的机器，例如 ubuntu-latest
步骤	作业中的一个任务，按顺序执行
动作	通过 uses: 引用的可复用步骤

2. 创建你的第一个工作流

在你现有的 cars-crud 仓库根目录中，创建：

```
cars-crud/
└── .github/
    └── workflows/
        └── hello-python.yml
```

将以下内容复制到 .github/workflows/hello-python.yml：

```yaml
name: Hello Python

on:
  push:
    branches: [main]
  workflow_dispatch:

permissions:
  contents: read

jobs:
  hello:
    runs-on: ubuntu-latest
```

```yaml
    steps:
      - name: Download the repository
        uses: actions/checkout@v7

      - name: Set up Python
        uses: actions/setup-python@v7
        with:
          python-version: "3.12"

      - name: Show Python version
        run: python --version

      - name: Print a message
        run: python -c "print('Hello from GitHub Actions!')"
```

每当有代码推送到 main 时，此工作流就会运行。当工作流文件存在于仓库的默认分支后，它也可以被手动启动。

3. 理解重要的 YAML 键

YAML 键	含义	可以修改它吗？
`name`	在 GitHub Actions 标签页中显示的名称	可以
`on`	启动工作流的事件	可以
`push`	在提交被推送后启动	可以：使用其他事件
`branches: [main]`	将此 push 触发器限制为 main	可以：修改分支名称
`workflow_dispatch`	允许手动执行	可选
`permissions`	设置工作流令牌的访问权限	可以；保持权限最小化
`jobs`	将工作分组为独立执行的作业	可以
`hello`	此作业的标识符	可以
`runs-on`	选择运行器的操作系统	可以
`steps`	作业内部按顺序执行的任务	可以
`uses`	运行一个可复用的动作，例如检出代码	可以：使用其他动作
`with`	向动作传递设置	取决于具体动作
`run`	在运行器的 shell 中执行命令	可以

actions/checkout@v7 会将你的仓库下载到运行器。actions/setup-python@v7 则会准备所选的 Python 版本。@v7 部分用于选择动作的主版本；升级动作时请查看其文档。

提示：YAML 缩进很重要。请使用空格，而不是制表符。多行命令可以写成 run: | 并在其后跟缩进的命令。

4. 触发事件与分支过滤

在 on: 下使用一个或多个事件（或组合多个事件）。

推送到某个分支之后：

```
on:
  push:
    branches: [main]
```

当 pull request 以 main 为目标时：

```
on:
  pull_request:
    branches: [main]
```

手动执行：

```
on:
  workflow_dispatch:
```

重要区别：

- 对于 push，branches 指定的是接收推送的分支。
- 对于 pull_request，branches 指定的是目标/基准分支，而不是源分支。例如，从 writing-tests 到 main 的 PR 会匹配 branches: [main]。
- pull_request 工作流通常在 PR 被打开、重新打开或添加新提交更新时运行。
- 一旦该工作流文件位于默认分支（通常是 main）上，workflow_dispatch 就会通过 GitHub 界面变得可用。

5. 将示例推送到 GitHub

确保你位于 cars-crud/ 目录中，且仓库已配置了 origin 远程。如果 main 分支未受保护，请使用：

```
git switch main
git pull --ff-only origin main
```

```
git add .github/workflows/hello-python.yml
git commit -m "Add first GitHub Actions workflow"
git push origin main
```

如果不允许直接推送到 main，请在单独的分支上提交、打开 pull request 并将其合并到 main。

6. 运行它并检查其状态

1. 打开你的 GitHub 仓库并选择 Actions 标签页。
2. 选择 Hello Python。将文件推送到 main 后应当会自动启动一次运行。
3. 打开最新的一次运行并选择 hello 作业。
4. 展开各个步骤，在日志中找到 Python 版本和 Hello from GitHub Actions!。
5. 寻找绿色对勾（成功）或红色叉号（失败）。运行中的作业会显示进行中的指示器。
6. 要再次手动运行，请选择 Hello Python > Run workflow，选择 main 并运行。

可选实验：将最后一个 run: 命令改为 python -c "raise SystemExit(1)"，提交并推送。检查失败的步骤，然后恢复原始命令并再次推送。

7. 将工作流适配到另一个项目

整体 YAML 结构保持不变。通常你只需修改以下部分：

部分	本 Python 示例	其他可能
运行器	`ubuntu-latest`	`windows-latest`, `macos-latest`
语言设置	`actions/setup-python`	`actions/setup-node`, `actions/setup-java`
版本	`python-version: "3.12"`	合适的 Node.js 或 Java 版本
依赖	`python -m pip install -r requirements-dev.txt`	`npm ci`、`mvn ...`、其他包工具
测试命令	`python -m pytest -q`	`npm test`, `mvn test`, other test 运行器

对于 Cars CRUD 项目，依赖和测试命令将在 GITHUB_ACTIONS_EXERCISE.md 中添加。上面的第一个示例故意只打印一条消息，而不是运行应用测试。

8. 常见问题

问题	检查
没有工作流运行	文件是否已推送到 .github/workflows/ 中？事件和分支是否匹配？Actions 是否已启用？
找不到 Run workflow	workflow_dispatch 是否已配置，且 YAML 文件是否位于默认分支上？

问题	检查
无效的工作流	检查 YAML 的空格、缩进和键。
找不到 Python 命令	检查运行器和 setup-python 步骤。
缺少包	在命令之前添加相应的依赖安装步骤。
作业失败	打开该次运行，点击失败的步骤，并阅读命令的输出。

官方参考

- GitHub：工作流语法
- GitHub：触发工作流的事件
- actions/checkout
- actions/setup-python